Módulo extra: Banco de dados e SQL

Sumário

Sumário..... 2
1. Fundamentos; Dados, Tabelas e propósito..... 3
1.1. O que são Dados?..... 3
1.2. O que é uma Tabela?..... 3
1.3. Para que usamos um Banco de Dados?..... 4
2. Capítulo 2: Chaves Primárias e Estrangeiras – Conectando o Mundo dos Dados..... 5
2.1. O que é uma Chave Primária (Primary Key - PK)?..... 5
2.2. O que é uma Chave Estrangeira (Foreign Key - FK)?..... 6
2.3. Por que as Chaves são Cruciais?..... 7
3. Introdução ao SQL – Criando e Consultando o seu Banco de Dados..... 8
3.1. DDL: Criando a Estrutura com CREATE TABLE..... 8
3.2. DML: Inserindo Dados com INSERT INTO..... 9
3.3. Consultando Dados com SELECT e WHERE..... 9
3.4. Unindo Tabelas com JOIN..... 10
4. Manutenção de Dados – Atualização (UPDATE) e Exclusão (DELETE)..... 10
4.1. Atualizando Registros com UPDATE..... 10
4.2. Excluindo Registros com DELETE..... 11
4.3. O Perigo das Chaves Estrangeiras na Exclusão..... 11
5. Agregação e Agrupamento – Extraindo Inteligência dos Dados (COUNT, SUM, GROUP BY)..... 12
5.1. Funções de Agregação (Resumindo Valores)..... 12
5.2. Agrupando Dados com GROUP BY..... 13
5.3. Filtrando Grupos com HAVING..... 13

1. Fundamentos; Dados, Tabelas e propósito

Para construir sistemas eficientes, escaláveis e robustos, precisamos primeiro dominar a base de como a informação é representada, estruturada e gerenciada.

Neste primeiro capítulo, vamos desvendar os três conceitos fundamentais que sustentam toda a arquitetura de dados: o que são dados, como se estruturam em tabelas e por que utilizamos sistemas de bancos de dados.

1.1. O que são Dados?

Dado Bruto: É um fato isolado, um valor ou uma observação sem contexto imediato. Por si só, o dado bruto não nos diz muita coisa.

exemplo: Gabriel, 29, Curitiba.

Sozinhos, esses valores são apenas sequências de caracteres ou números. No entanto, quando aplicamos um contexto, transformamos esses dados em informação útil.

exemplo: "O usuário Gabriel, nascido em 29 de novembro, reside em Curitiba."

Os bancos de dados existem justamente para capturar esses pedaços isolados de realidade, armazená-los de forma organizada e permitir que aplicações os transformem em conhecimento acionável.

1.2. O que é uma Tabela?

Em bancos de dados relacionais (o modelo mais tradicional e amplamente utilizado), a informação é organizada em tabelas (também chamadas de relações).

Pense em uma tabela como uma matriz bidimensional, estruturada de forma semelhante a uma planilha, mas com regras rígidas de tipagem e integridade. Ela é composta por:

- Colunas (Atributos ou Campos): Definem o tipo de dado que será armazenado e representam as características da entidade. Cada coluna possui um nome e um tipo de dado específico (ex: TEXT, INT, DATE).

- Linhas (Registros ou Tuplas): Representam uma instância única da entidade. Cada linha contém os valores correspondentes para cada coluna.

Exemplo

id_usuario (INT)	**nome (VARCHAR)**	**data_nascimento (DATE)**	**cidade (VARCHAR)**
1	Gabriel	29/11/2002	Curitiba
2	Luiz	03/05/2005	São Paulo
3	Rafael	12/08/2004	Londrina

Cada linha guarda o registro completo de um usuário específico, mantendo a padronização e garantindo que o sistema saiba exatamente onde buscar ou alterar a informação.

1.3. Para que usamos um Banco de Dados?

Muitos iniciantes se perguntam: "Por que não salvar tudo em arquivos de texto simples (como .txt, .csv) ou planilhas?"

Embora arquivos simples sirvam para casos triviais, eles falham miseravelmente quando a aplicação cresce. Utilizamos um Sistema de Gerenciamento de Banco de Dados (SGBD) pelas seguintes razões críticas:

- Persistência e Confiabilidade: Garantem que os dados não sejam perdidos em uma queda de energia ou reinicialização abrupta, mantendo-se salvos de forma permanente e segura.

- Concorrência: Permitem que milhares de usuários acessem, leiam e modifiquem os dados ao mesmo tempo, sem que ocorram conflitos ou corrupção de arquivos.

- Integridade: Asseguram que regras de negócio sejam respeitadas (por exemplo, impedindo que um usuário seja cadastrado sem um ID único ou com uma data inválida).

- Desempenho e Escalabilidade: Através de estruturas de indexação (como árvores B-Tree e hashes), os bancos encontram milésimos de segundos um registro específico em tabelas com milhões de linhas, algo inviável de varrer manualmente em arquivos de texto.

- Segurança e Controle de Acesso: Permitem definir exatamente quem pode ler, escrever ou modificar cada parte dos dados.

2. Capítulo 2: Chaves Primárias e Estrangeiras – Conectando o Mundo dos Dados

No Capítulo 1, vimos como os dados se organizam em tabelas formadas por linhas e colunas. No entanto, uma aplicação real nunca utiliza apenas uma tabela isolada. Em um sistema de e-commerce, por exemplo, temos tabelas para Clientes, Produtos, Pedidos e Pagamentos.

Como fazemos para que essas tabelas conversem entre si sem duplicar informações? É exatamente aqui que entram os pilares dos bancos de dados relacionais: as chaves primárias e as chaves estrangeiras.

2.1. O que é uma Chave Primária (Primary Key - PK)?

Toda tabela em um banco de dados relacional precisa de uma forma de identificar cada linha (registro) de maneira absolutamente única. Pense no seu CPF, no número de matrícula da faculdade ou no ID de um usuário: não podem existir duas pessoas com o mesmo CPF no sistema.

A Chave Primária (Primary Key) é a coluna (ou o conjunto de colunas) escolhida para cumprir esse papel. Ela possui duas regras fundamentais:

- Unicidade: Nenhum valor pode se repetir na tabela.

- Não-nulidade (Not Null): Toda linha obrigatoriamente precisa ter um valor válido nessa coluna (afinal, um registro sem identificador não pode existir).

exemplo: Tabela usuarios

Na tabela abaixo, a coluna id é a nossa Chave Primária. Note que o símbolo # ou uma chave costuma ser usado para indicá-la.

id (INT)	nome (VARCHAR)	email (VARCHAR)
1	Ronaldo	ronaldo@gmail.com
2	Evandro	evandro@gmail.com
3	Ana	ana@gmail.com

Se tentarmos inserir um novo usuário com o id = 2, o banco de dados rejeitará a operação imediatamente, protegendo a integridade da tabela contra duplicatas.

2.2. O que é uma Chave Estrangeira (Foreign Key - FK)?

Se as chaves primárias organizam cada tabela de forma isolada, as Chaves Estrangeiras (Foreign Keys) são responsáveis por conectar as tabelas entre si, criando o verdadeiro conceito de relacionamento.

Uma chave estrangeira é uma coluna (ou grupo de colunas) em uma tabela cuja função é apontar diretamente para a Chave Primária de outra tabela. Ela cria um elo lógico: o registro "filho" depende ou faz referência ao registro "pai".

Exemplo prático: Relacionamento entre usuarios e pedidos

Imagine que um usuário pode fazer vários pedidos. Não seria eficiente colocar o nome e o e-mail do usuário em cada pedido que ele fizer. Em vez disso, criamos duas tabelas separadas e as relacionamos:

Tabela usuarios (Tabela Pai)

id (INT)	nome (VARCHAR)
1	Gabriel
2	Ana

Tabela pedidos (Tabela Filha)

id_pedido (INT)	produto (VARCHAR)	id_usuario (INT)
100	Teclado Mecânico	1
101	Mouse Gamer	2
102	Monitor 144Hz	1

- Note que a coluna id_usuario na tabela pedidos é uma Chave Estrangeira.

- Ela aponta para o id da tabela usuarios.

- O pedido 100 e o 102 pertencem ao usuário 1 (Gabriel), enquanto o pedido 101 pertence à usuária 2 (Ana).

2.3. Por que as Chaves são Cruciais?

O uso correto de chaves primárias e estrangeiras traz benefícios indispensáveis para qualquer software:

- Integridade Referencial: O banco de dados impede que você crie um pedido associado a um usuário que não existe, ou que apague um usuário que ainda possui pedidos ativos (a menos que você configure regras de exclusão em cascata).

- Eliminação de Redundância: Em vez de repetir dados cadastrais em vários lugares, armazenamos a informação em um único lugar e usamos referências numéricas leves.

- Mapeamento do Mundo Real: Permite refletir estruturas complexas (como um sistema acadêmico, um ERP ou um banco de dados de jogos) de forma lógica, organizada e escalável.

3. Introdução ao SQL – Criando e Consultando o seu Banco de Dados

Agora que entendemos o que são dados, tabelas (Capítulo 1) e como usamos chaves primárias e estrangeiras para garantir a integridade e unicidade dos registros (Capítulo 2), chegou o momento de colocar a mão na massa!

Para interagir com um Banco de Dados Relacional, utilizamos a linguagem padrão de mercado: o SQL (Structured Query Language ou Linguagem de Consulta Estruturada). O SQL se divide em subconjuntos, sendo os mais importantes o DDL (para definir a estrutura das tabelas) e o DML (para manipular os dados lá dentro).

Vamos construir o nosso banco de dados passo a passo.

3.1. DDL: Criando a Estrutura com CREATE TABLE

Antes de guardar qualquer informação, precisamos estruturar as nossas tabelas, definindo seus campos, tipos de dados e as restrições (constraints) como as chaves primárias e estrangeiras que vimos antes.

Vamos criar duas tabelas para o nosso sistema: usuarios e pedidos.

```
- - Criando o banco de dados que você vai utilizar
CREATE DATABASE teste;

- - Utilizando o banco de dados que voce criou
USE teste;
-- Criando a tabela de usuários (Tabela Pai)
CREATE TABLE usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    data_cadastro DATE NOT NULL
);

-- Criando a tabela de pedidos (Tabela Filha com Chave
Estrangeira)
CREATE TABLE pedidos (
    id_pedido INT AUTO_INCREMENT PRIMARY KEY,
    produto VARCHAR(100) NOT NULL,
    valor DECIMAL(10, 2) NOT NULL,
    id_usuario INT NOT NULL,
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id)
);
```

- AUTO_INCREMENT: Diz ao banco para gerar o ID automaticamente somando +1 a cada novo registro.

- PRIMARY KEY: Define a coluna como a chave primária da tabela.

- FOREIGN KEY (...) REFERENCES ...(...): Estabelece o elo relacional, garantindo que nenhum pedido seja criado para um id_usuario que não exista na tabela usuarios.

3.2. DML: Inserindo Dados com INSERT INTO

Com as tabelas prontas, precisamos preenchê-las. É aqui que inserimos as linhas (registros).

```
-- Inserindo usuários
INSERT INTO usuarios (nome, email, data_cadastro) VALUES
('Gabriel Marques', 'gabriel@email.com', '2026-11-04'),
('Ana Souza', 'ana@email.com', '2026-11-04'),
('Carlos Lima', 'carlos@email.com', '2026-11-04');

-- Inserindo pedidos (note que usamos os IDs reais dos usuários
cadastrados)
INSERT INTO pedidos (produto, valor, id_usuario) VALUES
('Teclado Mecânico', 350.00, 1),
('Mouse Gamer', 180.00, 2),
('Monitor 144Hz', 1200.00, 1);
```

3.3. Consultando Dados com SELECT e WHERE

A operação mais comum em qualquer sistema é a consulta de dados. O comando SELECT busca as informações que precisamos.

- Buscando tudo de uma tabela:

```
SELECT * FROM usuarios;
```

- Filtrando resultados com WHERE: Se quisermos encontrar apenas os pedidos que custam mais de 300 reais:

```
SELECT produto, valor FROM pedidos WHERE valor > 300.00;
```

3.4. Unindo Tabelas com JOIN

Como os nossos dados estão divididos em tabelas diferentes (graças às chaves estrangeiras), em algum momento precisaremos juntá-los para exibir uma visão completa (por exemplo: "Qual usuário fez qual pedido?").

Para isso, usamos o INNER JOIN:

```
SELECT usuarios.nome, pedidos.produto, pedidos.valor
FROM pedidos
INNER JOIN usuarios ON pedidos.id_usuario = usuarios.id;
```

O que esse comando faz?

Ele cruza as linhas da tabela pedidos com a tabela usuarios sempre que o id_usuario do pedido for igual ao id da tabela de usuários, gerando um resultado combinado que mostra o nome da pessoa lado a lado com o produto que ela comprou.

4. Manutenção de Dados – Atualização (UPDATE) e Exclusão (DELETE)

Nos capítulos anteriores, aprendemos a criar a estrutura do banco de dados (Capítulo 1), a garantir a integridade com chaves (Capítulo 2) e a inserir e consultar informações usando SQL (Capítulo 3).

No entanto, o mundo real é dinâmico: e-mails mudam, produtos saem de linha, cadastros precisam ser corrigidos e registros antigos precisam ser limpos. Para gerenciar esse ciclo de vida dos dados, utilizamos dois comandos fundamentais do DML: UPDATE e DELETE.

4.1. Atualizando Registros com UPDATE

O comando UPDATE é utilizado para modificar dados que já existem em uma tabela.

- *Aviso de Segurança Crítica: Sempre utilize a cláusula WHERE em um comando UPDATE. Se você esquecer o WHERE, o SGBD vai alterar todos os registros da tabela de uma só vez!*

exemplo: Corrigindo ou alterando um e-mail

Imagine que o usuário Gabriel (cujo id é 1) mudou de e-mail. Para atualizar apenas o registro dele, fazemos:

```
UPDATE usuarios
SET email = 'novo.gabriel@email.com'
WHERE id = 1;
```

O que aconteceu aqui:

- UPDATE usuarios: Informa qual tabela queremos modificar.
- SET email = ...: Define qual coluna receberá o novo valor.
- WHERE id = 1: Garante que a alteração aconteça somente para a linha específica do Gabriel.

Se quisermos alterar mais de uma coluna ao mesmo tempo, basta separá-las por vírgula:

```
UPDATE usuarios
SET nome = 'Gabriel Marques da Silva', email =
'gabriel.silva@email.com'
WHERE id = 1;
```

4.2. Excluindo Registros com DELETE

O comando DELETE remove linhas inteiras de uma tabela. Assim como no UPDATE, a regra de ouro aqui é a mesma: nunca esqueça o WHERE, a menos que você realmente queira apagar todos os dados da tabela.

exemplo: Removendo um pedido cancelado

Imagine que o pedido com id_pedido = 2 (o Mouse Gamer) foi cancelado e precisa ser apagado do banco de dados:

```
DELETE FROM pedidos
WHERE id_pedido = 2;
```

4.3. O Perigo das Chaves Estrangeiras na Exclusão

Lembra das chaves estrangeiras que estudamos? Elas protegem a integridade do seu banco, mas podem gerar um bloqueio interessante na hora de usar o DELETE.

Se você tentar apagar um usuário que ainda possui pedidos vinculados na tabela pedidos, o banco de dados vai recusar a operação e retornar um erro (violação de integridade referencial). Afinal, o sistema não quer deixar um pedido "órfão" apontando para um usuário que não existe mais.

Para lidar com isso, existem duas abordagens principais na modelagem:

- Exclusão em Cascata `ON DELETE CASCADE`: Configura a tabela para que, ao apagar o usuário, todos os pedidos dele sejam apagados automaticamente.

- Exclusão Lógica (Soft Delete): Em vez de usar DELETE, muitas empresas apenas atualizam uma coluna de status (ex: ativo = 0 ou status = 'cancelado') usando um UPDATE, preservando o histórico para auditoria.

5. Agregação e Agrupamento – Extraindo Inteligência dos Dados (COUNT, SUM, GROUP BY)

Nos capítulos anteriores, aprendemos a criar tabelas, relacioná-las por chaves, inserir, consultar, atualizar e excluir registros individualmente. Mas e quando precisamos responder a perguntas de negócio complexas?

- "Qual é o faturamento total da nossa loja este mês?"
- "Quantos pedidos cada usuário realizou?"
- "Qual é a média de preço dos produtos por categoria?"

Para ir além de simples listagens e começar a gerar métricas, relatórios e inteligência, utilizamos as Funções de Agregação em conjunto com o comando GROUP BY.

5.1. Funções de Agregação (Resumindo Valores)

As funções de agregação pegam um conjunto de várias linhas e as condensam em um único valor calculado. As principais funções que você vai utilizar no dia a dia são:

- COUNT(): Conta quantas linhas atendem a um critério.
- SUM(): Soma os valores numéricos de uma coluna.
- AVG(): Calcula a média aritmética de uma coluna.
- MAX() e MIN(): Encontram o maior e o menor valor, respectivamente.

exemplo:

Quantos pedidos temos ao todo no sistema?

```
SELECT COUNT(*) AS total_pedidos FROM pedidos;
```

(O AS total_pedidos cria um apelido amigável para a coluna de resultado).

Qual é o valor somado de todos os pedidos?

```
SELECT SUM(valor) AS faturamento_total FROM pedidos;
```

5.2. Agrupando Dados com GROUP BY

Geralmente, não queremos apenas a soma ou a média de toda a tabela de uma vez só. Queremos que esses cálculos sejam divididos por categorias, por usuário, por mês ou por qualquer outro grupo. É aí que entra o GROUP BY.

O GROUP BY agrupa linhas que possuem o mesmo valor em colunas específicas, permitindo que a função de agregação seja aplicada separadamente a cada bloco.

exemplo: Quanto cada usuário gastou no total?

Lembra que na nossa tabela pedidos temos uma coluna id_usuario? Queremos saber o valor gasto por cada usuário individualmente.

```
SELECT id_usuario, SUM(valor) AS total_gasto
FROM pedidos
GROUP BY id_usuario;
```

O banco de dados vai:

- Olhar todos os pedidos.
- Agrupá-los separando-os por cada id_usuario distinto.
- Somar o valor de cada grupo separadamente.

O resultado será algo parecido com isto:

id_usuario	total_gasto
1	1550.00
2	180.00

5.3. Filtrando Grupos com HAVING

Aqui vai uma pegadinha muito comum em entrevistas e provas de banco de dados: você não pode usar a cláusula WHERE para filtrar o resultado de uma função de agregação.

- O WHERE filtra as linhas antes de elas serem agrupadas.

- O HAVING filtra os grupos depois que a agregação já aconteceu.

exemplo: Exibir apenas os usuários que gastaram mais de R$ 500,00 no total

```
SELECT id_usuario, SUM(valor) AS total_gasto
FROM pedidos
GROUP BY id_usuario
HAVING SUM(valor) > 500.00;
```

Se tentássemos usar WHERE SUM(valor) > 500.00, o SGBD retornaria um erro de sintaxe, pois a soma matemática ainda não havia sido calculada no momento em que o WHERE opera.